GießWerk: Modulares System

Nov 3, 2026 · @M

Kernprinzip

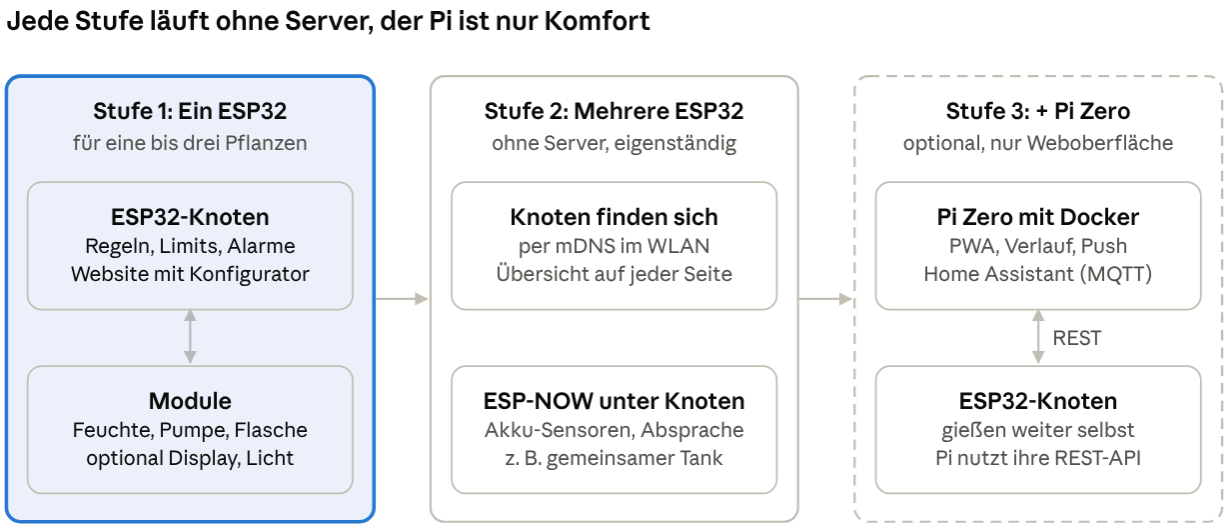
- Jeder ESP32 = vollständiges Gerät: gießt, speichert Konfiguration, schickt Alarme, hat eigene einfache Website + REST-API
- Mehrere ESP32 arbeiten zusammen – ohne Server
- Pi Zero 2 W optional, **nur Webserver** (Komfort-PWA, Langzeitverlauf, Push, Home Assistant) – keine Gießlogik
- Sensoren und Aktoren als austauschbare Module → anderer Hersteller = Konfiguration oder kleiner Treiber, keine Logikänderung
- Eine zentrale Schnittstelle: REST-API der Knoten, genutzt von Knoten-Website und Pi

Austauschbar und auf dem ESP32 konfigurierbar

- Eine Firmware für alle Knoten – was ein Knoten kann, steht nur in seiner Konfiguration; kein Neukompilieren, kein Code ändern
- Alles über die Knoten-Website einstellbar, ohne Pi:
  - Module: welcher Sensor/Aktor, welcher Treiber, Pin bzw. Bus und Adresse
  - Kalibrierung je Sensor (trocken/nass) und Fördermenge je Pumpe (ml/s)
  - Zonen: Pflanze ↔ Sensor ↔ Pumpe/Ventil
  - Regeln und Profile: Grenzwerte, Portionsgröße, Einsickerpause, Zeitpläne
  - Sicherheitslimits: max. Pumpzeit pro Vorgang und Tag
  - Alarme: Ziel (ntfy/Telegram), Schwellen, Ruhezeiten
  - Netzwerk: WLAN, Knotenname (mDNS), optional MQTT-Broker
  - Display-Seiten, Stromart (Kabel/Akku) und Messintervall
- Austauschbar ohne Änderung der Gießlogik:
  - Gleicher Sensortyp, anderer Hersteller → nur neue Kalibrierwerte
  - Andere Schnittstelle (analog ↔ I²C ↔ RS485) → anderer Treiber-Typ in der Konfiguration
  - Ganz neues Bauteil → ein neuer Treiber (eine Datei) + Eintrag im JSON Schema
  - Aktoren: Tauchpumpe, Peristaltikpumpe, Magnetventil – alle hinter „ein/aus“, Menge über kalibrierte Fördermenge
  - Board: ESP32-C6, S3 oder klassisch – Pinliste je Board in der Firmware, Konfigurator prüft Pinfähigkeiten

- Speicherung: JSON im Flash des ESP32, beim Speichern gegen das Schema geprüft; ungültige Konfiguration → letzte gültige bleibt aktiv
- Export/Import als Datei, „von Knoten X kopieren“; der Pi zeigt nur an und schickt Änderungen an den Knoten zurück
- Beispiel Sensortausch: Modultyp im Konfigurator ändern → Bus/Adresse wählen → trocken/nass kalibrieren → speichern, fertig

Ausbaustufen



Ausbaustufen · Pi optional

Pi ausstecken → System läuft auf Stufe 2 weiter.

Ideen

Modulsystem

- Drei Ebenen: **Treiber** (Rohwert lesen) → **Messgröße** (Einheit, Kalibrierung) → **Logik** (kennt nur Messgrößen, nie Pins/Hersteller)
- Treiber-Typen z. B. `analog_capacitive`, `i2c_stemma`, `bh1750`, `sht40`, `mosfet`; später `modbus_rs485`
- Welches Modul an welchem Pin: JSON-Konfiguration statt Code
- Konfigurationsformat als **JSON Schema** im Repo → gemeinsamer Vertrag Firmware ↔ Webapp
- Kalibrierung: 2 Punkte (trocken/nass), linear, auf 0–100 % begrenzt
- Neuer Sensortyp: Treiber + Registereintrag + Schema-Eintrag + Katalogseite, Ziel < 1 Tag

Sensoren und Regeln

- Pflicht: Bodenfeuchte; optional: Luftfeuchte, Temperatur, Helligkeit, Tankfüllstand, Durchfluss
- System bietet nur Funktionen an, für die Sensoren vorhanden sind
- Ohne Sensor → Zeitsteuerung · + Feuchte → Schwellwert, Portionen mit Einsickerpause · + Temp/Luftfeuchte → Verdunstung berücksichtigen · + Licht → nicht in praller Sonne gießen · + Füllstand → „reicht bis …"-Prognose
- Sensorausfall erkennen (Wert außerhalb Bereich, eingefroren, steigt nach Gießen nicht) → Rückfall auf nächstniedrigere Stufe + Alarm
- Harte Sicherheitslimits (max. Pumpzeit pro Vorgang/Tag) immer aktiv

Konfigurator

- Läuft immer auf der Knoten-Website; PWA auf dem Pi = gleicher Ablauf, bequemer, alle Knoten
- 5 Schritte: Board wählen → Module + Pins (Konfliktprüfung, z. B. ADC2 bei WLAN) → Verdrahtungsplan → Live-Kalibrierung → Pflanzen/Zonen/Regeln
- Formulare aus dem JSON Schema generiert → neuer Treiber ohne Frontend-Änderung
- Konfiguration liegt immer auf dem ESP32 (maßgebliche Kopie)

Mehrere ESP32 ohne Server

- Alle Knoten gleichberechtigt, kein Master
- Finden per mDNS (`giesswerk-kueche.local`); Fallback: feste IP + QR-Code am Gerät
- Jede Knoten-Website zeigt die anderen Knoten mit Status (Browser fragt deren API direkt)
- Absprache per ESP-NOW: Akku-Sensoren → Pumpenknoten, gemeinsamer Tank
- Alarme direkt vom Knoten (ntfy, Telegram); ohne Internet: Summer/LED
- Kurzverlauf 24–72 h im RAM, Mini-Diagramm auf der Website
- Uhrzeit per NTP; ohne Internet RTC-Modul (DS3231)
- „Konfiguration von Knoten X kopieren"

Pi Zero: nur Webserver

- PWA (installierbar), Übersicht aller Knoten
- Holt Messwerte per REST-API, speichert Langzeitverlauf (SQLite)
- Web Push, Dünger-Erinnerung
- Optional Mosquitto für Home Assistant (MQTT Discovery)
- Schlanker Stack wegen 512 MB RAM: FastAPI/Go + SQLite, ARM64-Images

Bewässerung

- Tank: PET- oder Glasflasche, Deckel mit Bohrung
- Heber-Effekt vermeiden: Flasche unter Topfoberkante oder Belüftung/Rückschlagventil
- Füllstand: Wägezelle + HX711 (ml genau, misst auch Gießmenge)
- Tropfschlauch 4/6 mm, Löcher 0,5–1 mm, Ende zu, als Ring um die Pflanze; alternativ Micro-Drip
- Mehrere Töpfe: 1 Pumpe + Ventil je Topf, oder Peristaltikpumpe je Topf
- Dünger: Erinnerung (Software) → Dosierpumpe aus 2. Flasche (Ausbau) · nie in den Haupttank

Strom

- Pumpenknoten am Kabel (5 V USB-C oder 12 V + Step-Down)
- Sensorknoten am Akku (18650 + Lade-/Schutzmodul), Deep Sleep, Sensoren nur beim Messen versorgen
- Akkuspannung messen, Warnung bei niedrigem Stand
- Optional Solar für Balkon

Gehäuse und Display

- Variante A: 3D-Druck (PETG), STL + STEP veröffentlichen
- Variante B: ohne 3D-Druck – Abzweigdose/Holzkästchen, Holzrahmen für Glasflasche, Bohrschablone als PDF
- Gleiche Maße in beiden Varianten
- Display am ESP32 (nicht am Pi): OLED für Kabelknoten, E-Paper für Akkuknoten
- Display-Seiten im Konfigurator wählbar, Taster: kurz = blättern, lang = jetzt gießen

Teams

Team	Verantwortung	Liefert an die anderen
Hardware (2)	Sensoren testen und kalibrieren, Pumpen und Ventile auswählen, Schaltplan, Stromversorgung, Aufbau, 3D-Druck, Dauertest, Stückliste	Verifizierte Sensorwerte und Kalibrierdaten, Pinbelegung, Hardware-Doku

Team	Verantwortung	Liefert an die anderen
ESP32-Software (2)	Firmware, Gießlogik und Sicherheitslimits, WLAN und MQTT, Notfallprofil, OTA, Provisioning, ESP-NOW	MQTT-Topics und Nachrichtenformat, simulierter Knoten für die Web-Gruppe
Docker-Webapp (2)	Backend, Datenbank, Broker, Alarme, PWA, Home-Assistant-Anbindung, Docker-Compose-Setup	API und MQTT-Spec, Fake-Daten, Deployment

- Knoten-Website: **ESP32-Software liefert die REST-API, Webapp baut die Oberfläche** (kleines Preact/HTML-Bundle, das die Firmware ausliefert) → Web-Know-how bleibt bei der Webapp, ESP-Gruppe wird entlastet
- Mit KI ist die ESP32-Software die meistbelastete Gruppe (Embedded-Code profitiert am wenigsten) → nach dem MVP hilft eine Person aus der Hardware bei ESP-NOW und Akkubetrieb
- Schnittstellen in Sprint 1 als Dateien im Repo: REST-API (OpenAPI), JSON Schema, MQTT-Topics, Pinbelegung

Schätzung mit KI

Annahme: Alle nutzen Claude Code und GitHub Copilot. KI beschleunigt Code stark, aber nicht Hardware, Messungen und Fehlersuche am Gerät. Deshalb hat jede Story einen **KI-Hebel**, der in die Story Points eingeflossen ist.

KI-Hebel	Typische Arbeit	Effekt auf die Schätzung
hoch	Web-UI, REST-Endpunkte, Tests, Docker-Setup, Doku, reine Logik	ca. halbiert gegenüber ohne KI
mittel	Architektur, Schnittstellen, Firmware mit WLAN/HTTP, Integration	ca. ein Viertel weniger
gering	Löten, Messen, Kalibrieren, Funk/Strom debuggen, 3D-Druck, Dauertest	unverändert

Bezugsgröße: 1 SP ≈ 3–4 Stunden inkl. Review und Test auf echter Hardware.

Kapazität: 6 Personen × 5 h × ~27 Wochen ≈ 800 h; abzüglich ~20 % Meetings und Abstimmung ≈ 650 h ≈ **160–200 SP**. Das Backlog unten hat **143 SP** – Puffer für Integration, Bugs und Lernkurve. Must-Stories: 85 SP → bis Ende Semester 1 machbar.

Damit die KI gute Arbeit liefert:

- `CLAUDE.md` und `.github/copilot-instructions.md` im Repo: Architektur, Schnittstellen, Codestil, „keine Pins/Hersteller in der Logik“
- OpenAPI und JSON Schema zuerst schreiben – daraus lassen sich Client, Server-Stubs, Formulare und Tests generieren
- Embedded Rust: KI kennt oft veraltete APIs (esp-hal hat sich bis 1.0 stark verändert) → Doku-Links mitgeben, mit Beispielen aus dem eigenen Repo arbeiten
- Sicherheitslimits und Pumpensteuerung: KI-Code nie ungeprüft übernehmen, immer mit Test + Review
- Velocity nach Sprint 2 messen; liegt sie deutlich unter der Schätzung, KI-Hebel ehrlich korrigieren

Product Backlog

50 Stories, 143 SP, davon Must 85 SP. Teams: HW = Hardware, ESP = ESP32-Software, Web = Docker-Webapp.

Teil 1: Fundament, Hardware, Firmware, Knoten-Website

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
1	Fundament	Als Team wollen wir Repo, CI für Firmware und Webapp und ein Scrum-Board, damit alle sofort arbeiten können.	Must	2	Alle	hoch
2	Fundament	Als Team wollen wir REST-API der Knoten (OpenAPI), JSON Schema der Konfiguration und MQTT-Topics festlegen, damit alle parallel arbeiten.	Must	3	ESP, Web	mittel
3	Fundament	Als ESP-Gruppe wollen wir Rust und C++ auf dem ESP32-C6 in einem Spike vergleichen, um die Sprache fundiert zu wählen.	Must	3	ESP	gering
4	Fundament	Als Web-Gruppe wollen wir einen simulierten Knoten mit derselben REST-API, um ohne Hardware zu entwickeln.	Must	1	ESP	hoch
5	Hardware	Als Team wollen wir 3–4 Feuchtesensoren verschiedener Hersteller vergleichen und kalibrieren, um verlässliche Kalibrierdaten zu haben.	Must	5	HW	gering

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
6	Hardware	Als Nutzer will ich eine Flasche als Tank mit Pumpe und Tropfschlauch, die nach dem Gießen nicht weiterläuft.	Must	3	HW	gering
7	Hardware	Als Team wollen wir Schaltplan und Stückliste eines Knotens (MOSFET, Netzteil, Step-Down), damit jeder ihn nachbauen kann.	Must	3	HW	mittel
8	Hardware	Als Nutzer will ich die Restmenge im Tank in ml sehen (Wägezelle mit HX711).	Should	3	HW, ESP	gering
9	Hardware	Als Nutzer will ich Sensorknoten am Akku betreiben (18650, Lademodul, Spannungsmessung).	Should	3	HW	gering
10	Hardware	Als Nutzer will ich ein 3D-gedrucktes, spritzwassergeschütztes Gehäuse, dessen Dateien veröffentlicht sind.	Should	5	HW	gering
11	Hardware	Als Nutzer ohne 3D-Drucker will ich eine Bauanleitung mit Standardteilen und Bohrschablone.	Should	3	HW	gering
12	Hardware	Als Nutzer will ich Flüssigdünger über eine zweite Pumpe automatisch dosieren.	Could	3	HW, ESP	gering
13	Modulsystem	Als Entwickler will ich eine Treiber-Schnittstelle mit Register, damit Sensoren anderer Hersteller ohne Logikänderung einsetzbar sind.	Must	5	ESP	mittel
14	Modulsystem	Als Nutzer will ich Treiber für kapazitiven Feuchtesensor, BH1750, SHT40 und MOSFET-Aktor nutzen können.	Must	3	ESP	hoch
15	Modulsystem	Als Nutzer will ich Sensoren über zwei Messpunkte (trocken/nass) kalibrieren.	Must	2	ESP	hoch
16	Modulsystem	Als Nutzer will ich, dass die Konfiguration im Flash gespeichert und gegen das Schema geprüft wird.	Must	3	ESP	mittel
17	Gießlogik	Als Nutzer will ich, dass bei zu trockener Erde automatisch in Portionen mit Einsickerpause gegossen wird.	Must	3	ESP	hoch
18	Gießlogik	Als Nutzer will ich harte Sicherheitslimits und eine Tank-leer-Sperre, damit nie überschwemmt wird.	Must	2	ESP	hoch

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
19	Gießlogik	Als Nutzer will ich, dass nur Funktionen für vorhandene Sensoren angeboten werden und das System bei Sensorausfall sicher zurückfällt.	Must	5	ESP	mittel
20	Gießlogik	Als Nutzer will ich Zeitpläne, die auch ohne Internet stimmen (NTP, RTC-Fallback).	Should	2	ESP	hoch
21	Gießlogik	Als Nutzer will ich Pflanzenprofile (z. B. Kaktus, Basilikum) mit Startwerten wählen.	Should	2	Web	hoch
22	Knoten-Website	Als Entwickler will ich eine REST-API auf dem ESP32 für Status, Konfiguration und Gießen.	Must	3	ESP	mittel
23	Knoten-Website	Als Nutzer will ich einen neuen Knoten per WLAN-Hotspot vom Handy aus ins Heimnetz bringen.	Must	3	ESP	mittel
24	Knoten-Website	Als Nutzer will ich auf der Knoten-Website Status sehen und manuell gießen.	Must	2	Web	hoch
25	Knoten-Website	Als Nutzer will ich auf der Knoten-Website Module hinzufügen, Pins zuordnen und live kalibrieren.	Must	5	Web, ESP	hoch
26	Knoten-Website	Als Nutzer will ich einen Kurzverlauf der letzten 24–72 Stunden auf der Knoten-Website sehen.	Should	2	ESP	hoch
27	Knoten-Website	Als Nutzer ohne Server will ich Alarme direkt vom Knoten aufs Handy bekommen (ntfy).	Must	2	ESP	hoch
28	Knoten-Website	Als Nutzer will ich die Firmware per OTA aktualisieren.	Should	3	ESP	mittel
29	Knoten-Website	Als Nutzer will ich ein Display am Knoten mit wählbaren Seiten.	Could	3	ESP, HW	mittel

Teil 2: Mehrere Knoten, Pi-Webserver, Qualität, Doku

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
30	Mehrere Knoten	Als Nutzer will ich auf jeder Knoten-Website die anderen Knoten im Netz mit Status sehen (mDNS).	Must	3	ESP	mittel
31	Mehrere Knoten	Als Nutzer will ich Akku-Sensoren, die per ESP-NOW an einen Pumpenknoten melden.	Should	5	ESP, HW	gering

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
32	Mehrere Knoten	Als Nutzer will ich, dass Knoten an einem gemeinsamen Tank sich absprechen.	Could	3	ESP	gering
33	Mehrere Knoten	Als Nutzer will ich die Konfiguration eines Knotens auf einen anderen kopieren.	Could	1	ESP	hoch
34	Pi-Webserver	Als Nutzer will ich die Weboberfläche per Docker Compose auf einem Pi Zero 2 W starten.	Must	3	Web	mittel
35	Pi-Webserver	Als Nutzer will ich, dass der Pi alle Knoten findet, ihre Werte abholt und als Langzeitverlauf speichert.	Must	3	Web	hoch
36	Pi-Webserver	Als Nutzer will ich ein Dashboard aller Pflanzen mit Ampelstatus.	Must	3	Web	hoch
37	Pi-Webserver	Als Nutzer will ich Verlaufsdiagramme über Tage, Wochen und Monate.	Must	2	Web	hoch
38	Pi-Webserver	Als Nutzer will ich im Konfigurator der PWA alle Knoten bearbeiten, mit aus dem Schema generierten Formularen und Pin-Konfliktprüfung.	Should	5	Web	mittel
39	Pi-Webserver	Als Nutzer will ich die PWA auf dem Handy installieren (HTTPS im Heimnetz).	Should	3	Web	gering
40	Pi-Webserver	Als Nutzer will ich Web-Push-Benachrichtigungen.	Should	3	Web	mittel
41	Pi-Webserver	Als Home-Assistant-Nutzer will ich, dass meine Knoten automatisch in HA erscheinen (MQTT Discovery).	Should	3	Web, ESP	mittel
42	Pi-Webserver	Als Nutzer will ich ans Düngen erinnert werden, passend zum Pflanzenprofil.	Could	1	Web	hoch
43	Pi-Webserver	Als Nutzer will ich aus meiner Konfiguration einen Verdrahtungsplan erzeugen.	Could	2	Web	hoch
44	Qualität	Als Team wollen wir Unit-Tests der Gießlogik mit einem simulierten Topf im Zeitraffer.	Must	2	ESP	hoch
45	Qualität	Als Team wollen wir End-to-End-Tests der Weboberflächen (Playwright) in der CI.	Should	2	Web	hoch
46	Doku	Als Entwickler will ich einen Hardware-Katalog mit Anschluss, Kalibrierung und Stolperfallen je Modul.	Must	3	HW	mittel

ID	Epic	User Story	Prio	SP	Team	KI-Hebel
47	Doku	Als Endanwender will ich eine bebilderte Aufbau- und Bedienungsanleitung.	Must	3	Alle	mittel
48	Doku	Als Entwickler will ich Architektur, API und „neuen Treiber hinzufügen“ dokumentiert vorfinden.	Must	2	Alle	hoch
49	Qualität	Als Team wollen wir einen 14-Tage-Dauertest mit echten Pflanzen durchführen und protokollieren.	Must	3	HW	gering
50	Qualität	Als Prüfer will ich in der Abschluss-Demo sehen, dass alles weitergießt, wenn der Pi ausgesteckt wird.	Should	1	Alle	gering

Bewusst nicht im Scope: native Apps, Cloud-Hosting, EC-Sensor für Dünger, Kamera-Pflanzenerkennung, eigene Platine in Serienqualität.